

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Installing Linux

Unit 1 • Linux (10-151-105)



Learning Outcomes

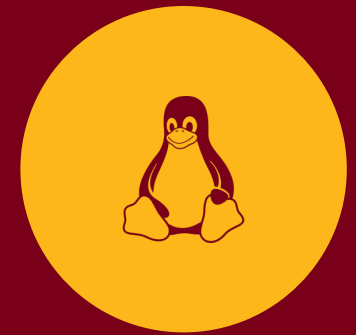
We begin with Linux fundamentals, then work toward this unit's core competency — installing a Linux operating system onto a host. By the end of the week you will be able to:

1. Verify that a host meets the minimum hardware requirements for a selected Linux operating system.
2. Apply settings during the installation process to align with user needs.
3. Modify operating system storage directories based on user needs.
4. Install the device drivers necessary for the computer to operate.

Along the way you'll also build the everyday command-line skills every Linux administrator relies on.

LESSON 1

Linux Foundations



Why Linux?

Where Linux runs

- Most public web servers and cloud instances
- Nearly all of the world's fastest supercomputers
- Containers, virtual machines, and app platforms
- Android phones, routers, firewalls, embedded devices

Why it's strong for servers

- Stability — months or years of uptime
- Efficient — runs well on modest hardware, no GUI overhead
- Secure — strong permissions, fast patching, SELinux/AppArmor
- Free — no per-server licensing for most distributions
- Scriptable — command-line first and automation-friendly
- Remote — fully managed over SSH

Common server roles — web, DNS, file, print, log, database, and the VMs and containers behind modern infrastructure — typically run on Linux.

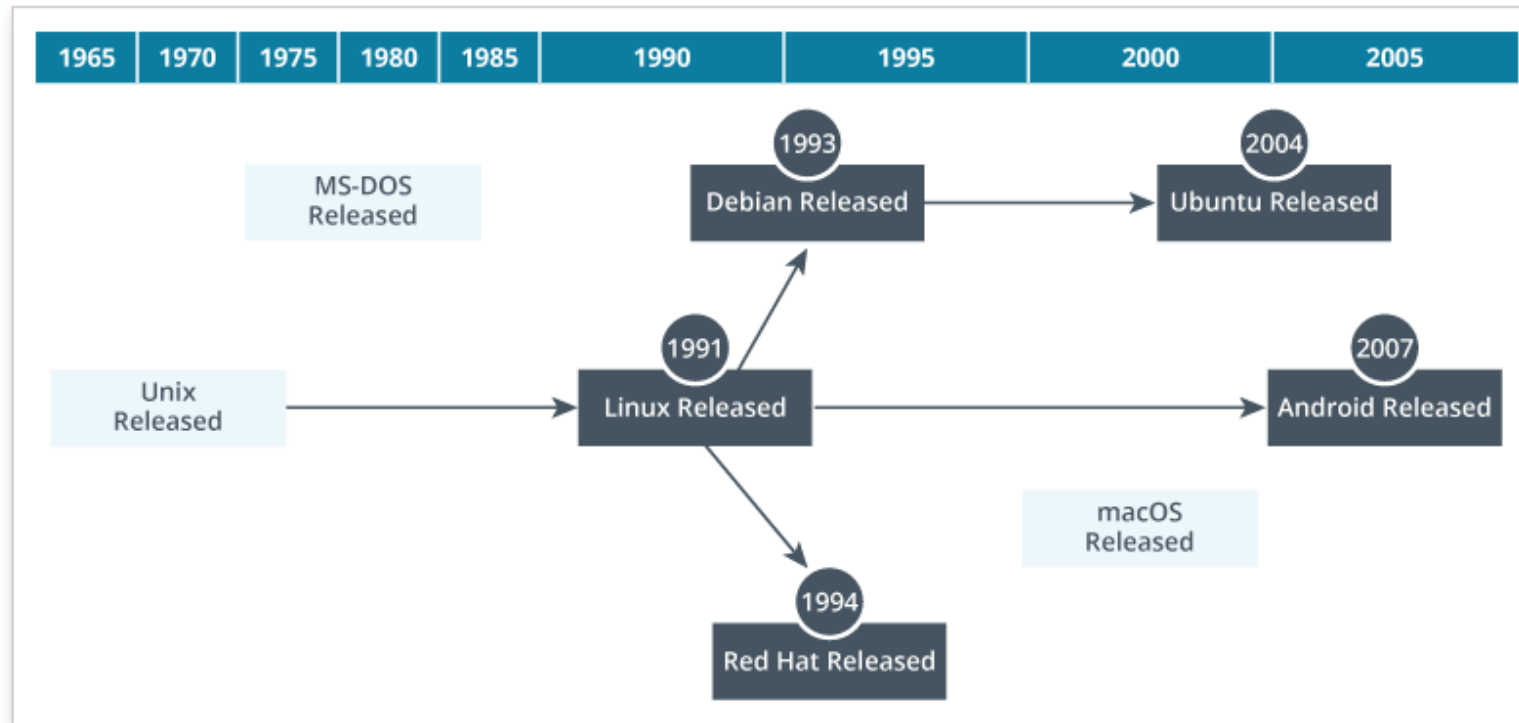
The Two Families: Debian & RHEL

	Debian family	Red Hat (RHEL) family
Origin	Community project, founded 1993	Commercial — Red Hat, Inc.
Package format	.deb	.rpm
Package manager	apt / dpkg	dnf / rpm (yum)
Firewall / security	ufw / AppArmor	firewalld / SELinux
Common members	Ubuntu, Linux Mint, Kali	RHEL, Rocky Linux, AlmaLinux, Fedora
In this course	Ubuntu Server 24.04	Rocky Linux 10

Most commands are the same across distributions; software management is the big difference — apt/dpkg on Debian, dnf/rpm on Red Hat.

A Short History of Linux

After Linux arrived in 1991, two major branches — Debian and Red Hat — followed quickly and spawned hundreds of distributions:



Unix (1969) set the foundation; the GNU Project (1983) and Linus Torvalds' kernel (1991) made a free OS possible; Ubuntu (2004) made it approachable.

Watch: Linux — The Untold Story

This short documentary traces how Linux came to be — from Unix and the GNU Project to Linus Torvalds' kernel and the rise of Ubuntu — and the culture that still shapes the system today.

▶ [Watch on Panopto](#)

VIDEO

Opens in your browser · closed captions available.

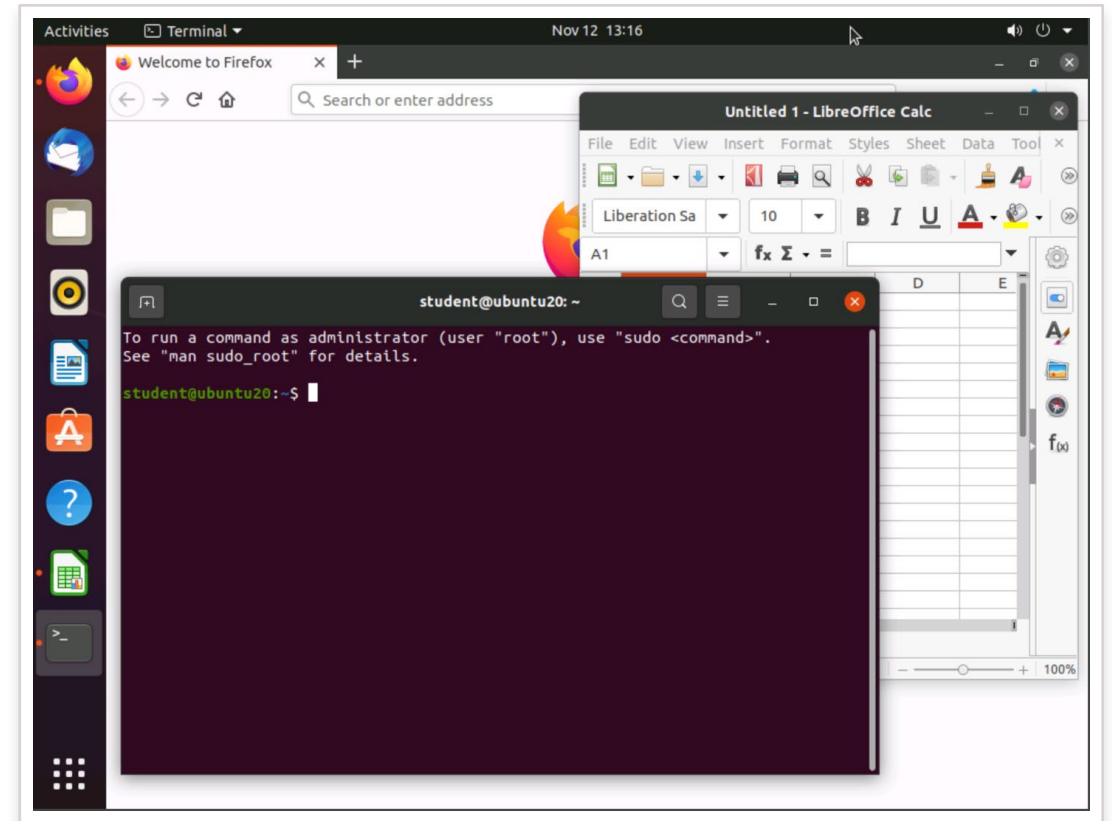
LESSON 2

Working in Linux



Linux Interfaces: CLI vs. GUI

- Linux is managed mainly from the command-line interface (CLI).
- A GUI is optional — and often skipped on servers to save resources and reduce risk.
- CLI strengths: speed, performance, and scriptability.
- Trade-offs: commands can be non-intuitive and inconsistent.
- Many desktop environments exist: GNOME, KDE Plasma, Cinnamon, MATE.



The Shell & Bash

- A shell provides the CLI: it takes your input, processes it, and returns output.
- Bash is the default shell on most Linux distributions — the one to know for Linux+.
- Other shells: ksh (Unix), Zsh (powerful scripting, default on macOS), Fish (friendly).
- Linux holds its configuration in text files, so editing those files is how you change settings.

Bash Command Syntax

Bash commands follow a structure. Each part has a name:

- Command — the primary instruction.
- Subcommand — a secondary, more detailed instruction.
- Option — a modifier, usually after a dash (-).
- Argument — the object the command acts on.

Examples

```
ls -la /var/log  
ip addr show eth0  
ip link help
```

Components are case-sensitive and separated by spaces; a wrong command returns a descriptive error — read it.

Essential Commands

Command	What it does	Example
pwd	Print working directory	pwd
ls	List directory contents	ls -la /var/log
cd	Change directory	cd /etc
cat	Display a file's contents	cat data.txt
less	View a file one screen at a time	less data.txt
touch	Create an empty file / update timestamp	touch newfile.txt
tree	Show directory structure as a tree	tree /etc
whoami	Show the current user	whoami

LIVE DEMO

On the Ubuntu box: pwd, ls -la, cd /etc, cat, whoami.

Working Efficiently at the CLI

- Tab completion — type part of a command or file name and press Tab to finish it — faster, fewer typos.
- Command history — press the Up / Down arrows to recall and reuse previous commands.
- Re-run from history — type history to list past commands, then !<number> to run one again. Bash keeps history across reboots.
- clear — clears the screen so you can start a task without distraction.
- Read it backward — when a command fails, read right-to-left to spot a missing or extra character.

LIVE DEMO

Show Tab completion, Up-arrow recall, history + !n, and clear.

Getting Help: man Pages

- man pages are the primary reference for commands — syntax, options, and examples.
- Syntax: `man <command>` (for example, `man ls`).
- Sections include NAME, SYNOPSIS, DESCRIPTION (with options), and SEE ALSO.
- Navigate: `/` to search, `n` / `p` for next / previous match, `q` to quit.
- Also handy: `<command> --help` or `-h` for a quick reference, and `whatis <command>` for a one-line summary.
- Vendor docs (Red Hat, Ubuntu) and the Linux Documentation Project (tldp.org) round out your options.

LIVE DEMO

Open `man ls`, search with `/sort`, press `q`; then `ls --help` and `whatis ls`.

Editing Config Files: nano & vim

Configuration lives in text files, so every admin edits them from the CLI. Two editors are always worth knowing:

nano

Simple. Just type; **Ctrl+O** saves, **Ctrl+X** exits.

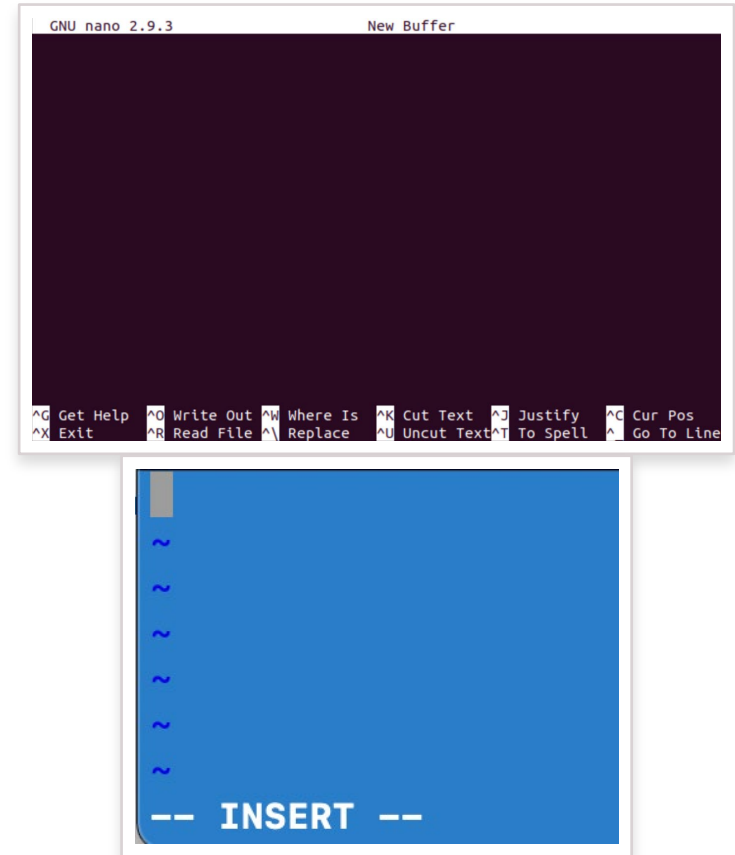
vim

Powerful, modal. **i** to insert, **Esc** then **:wq** to save and quit.

Know both at a basic level — open, edit, save, close — so you can use whatever a system has.

LIVE DEMO

Edit a file in nano (Ctrl+O, Ctrl+X), then in vim (i ... Esc ... :wq).



Privilege Elevation: su & sudo

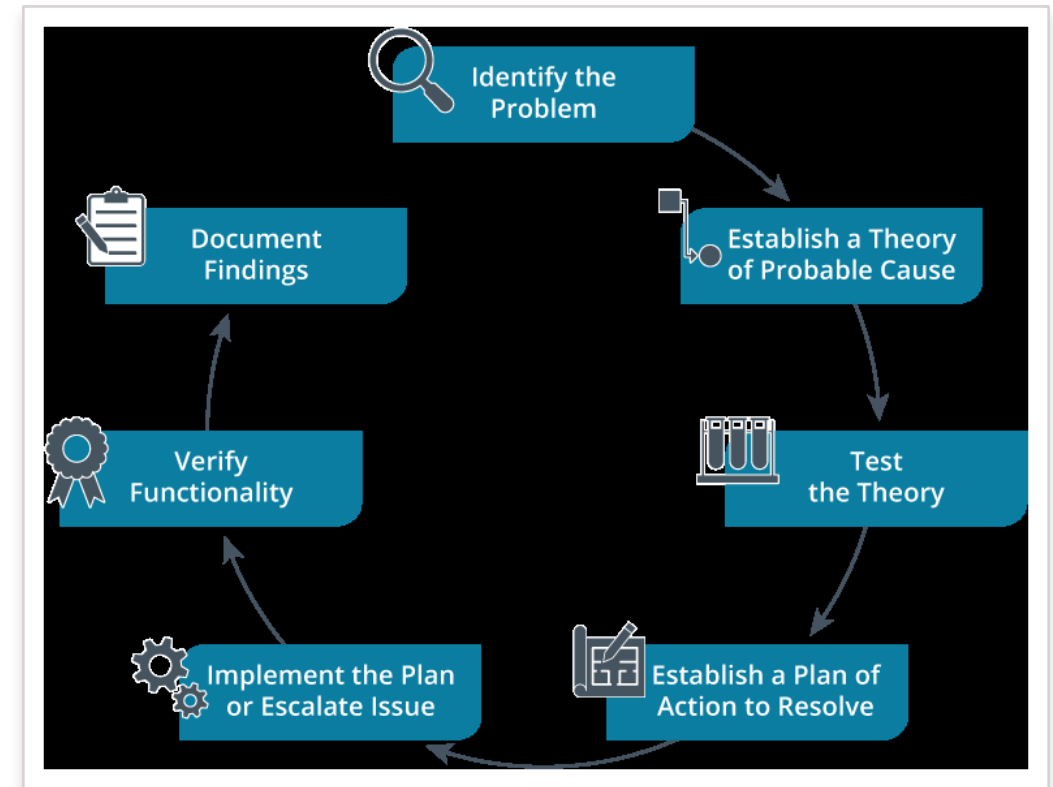
- Three account types: root (the administrator), standard user, and service accounts.
- Best practice: log in as a standard user and elevate only when needed — never live as root.
- `su` - switches to another user (`su - root` for a full root login); exit returns.
- `sudo` runs a single command as root — safer than a full root session.
- `/etc/sudoers` delegates specific tasks to specific users — the principle of least privilege.
- Many distributions disable the root login outright and require `sudo`.

LIVE DEMO

Run a command with `sudo`, then try `su -` and confirm with `whoami`.

The Troubleshooting Methodology

- Identify the problem (users, logs, alerts) — what changed?
- Determine the scope — one user or many?
- Establish and test a theory of probable cause — start simple.
- Establish a plan of action; implement it or escalate.
- Verify functionality, then prevent recurrence.
- Document findings throughout — not just at the end.



LESSON 3

Preparing to Install



Verify Hardware Requirements

Before installing, confirm the host can run your chosen distribution:

- CPU — 64-bit (x86-64 or ARM64); check virtualization support (VT-x / AMD-V) if it will host VMs.
- RAM — 1–2 GB minimum for a server; more for a GUI or heavier services. Plan for the workload.
- Disk — roughly 10–20 GB minimum; size for the OS, swap, data, and future growth.
- Firmware — UEFI (modern, faster, Secure Boot) or legacy BIOS — know which the host uses.
- Compatibility — confirm the network card, storage controller, and GPU are supported.

LIVE DEMO

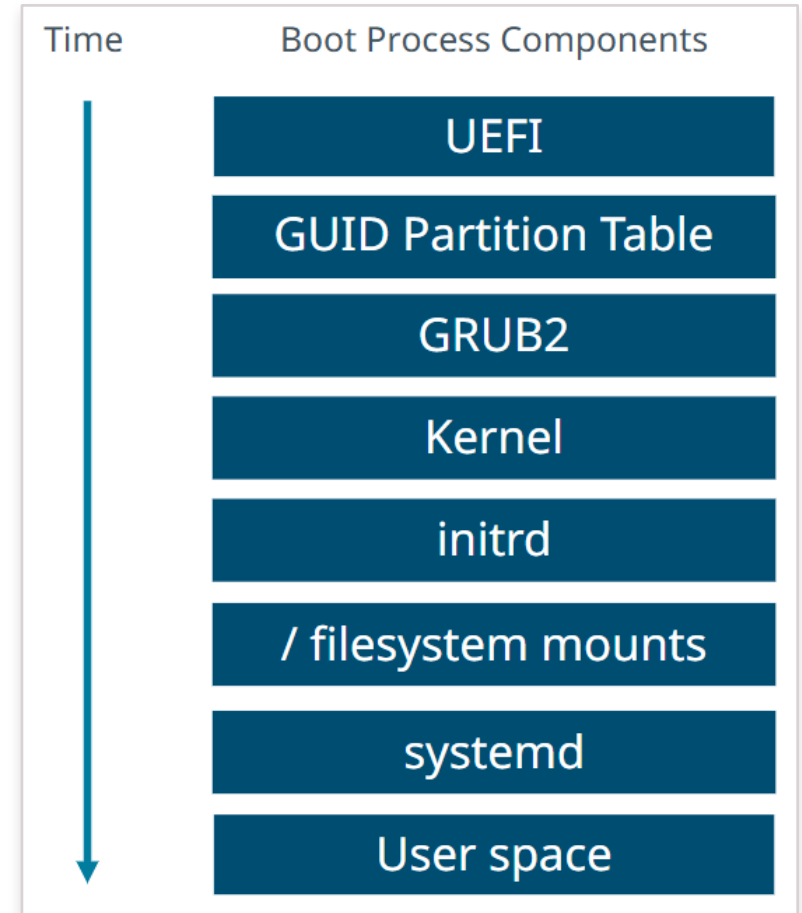
Check the Ubuntu box: `lscpu`, `free -h`, `lsblk`, `lspci`.

Installation Media & Sources

- Most distributions ship as an ISO image you download from the vendor.
- Always choose the latest release for current security fixes and hardware support.
- Verify the download with a checksum or signature before you use it.
- Install sources: bootable USB, DVD, an ISO mounted to a VM, or network installs over PXE, HTTP/FTP, or NFS.
- In this course you'll install inside a virtual machine in NETLAB+ from a provided ISO.

Firmware & Boot Basics

- Firmware runs first — UEFI on modern systems (faster, Secure Boot) or legacy BIOS on older ones.
- Boot order — decides which device the firmware starts from; point it at your install media.
- Bootloader (GRUB2) — loads the kernel and lets you pick an OS or kernel version at startup.
- Secure Boot — validates the bootloader and kernel signatures to block tampered boot code.



LESSON 4

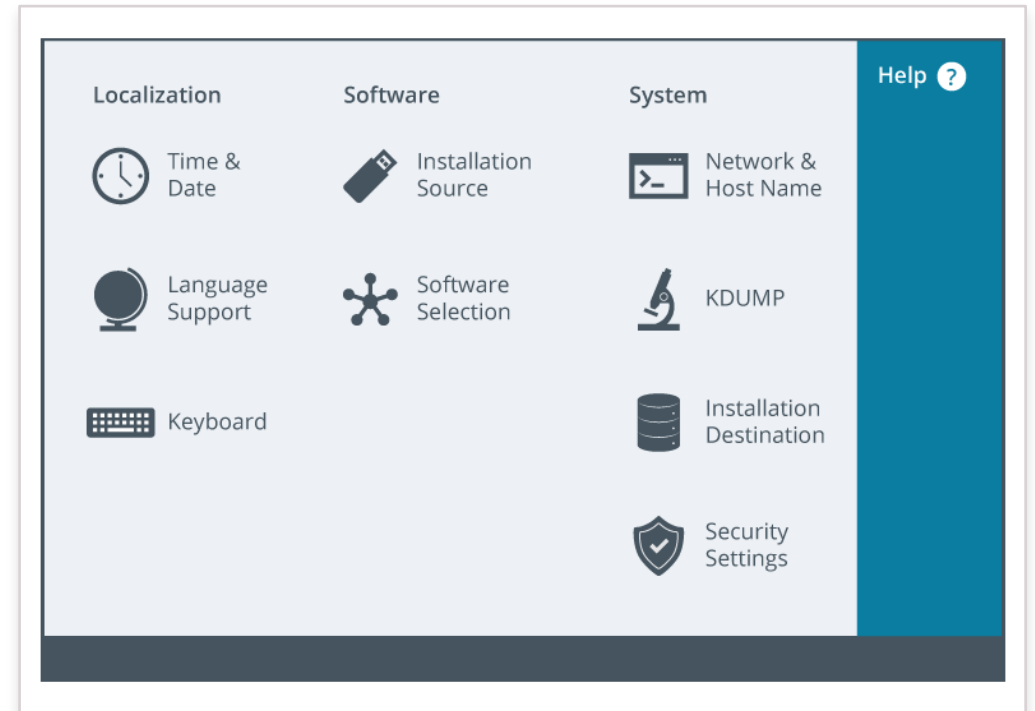
The Installation Process



The Installation Process

Most installers walk the same sequence — here it is on Rocky's Anaconda installer:

1. Boot the installer from your media.
2. Set language, keyboard, and locale.
3. Set date, time, and time zone.
4. Choose the installation source and destination disk.
5. Plan storage — partitions and mount points.
6. Select the system role and software packages.
7. Set a root password; create a non-root admin user.
8. Configure network settings (hostname, IP).
9. Install, reboot, and finish post-install tasks.



Apply Settings: Locale, Network & Time

- Language & keyboard — match the user's locale; drives input and how dates and numbers display.
- Date, time & time zone — an accurate clock is critical for logs, scheduled jobs, and security certificates.
- Hostname — give the system a clear, meaningful name that fits your naming scheme.
- Network — choose a static IP or DHCP — servers usually use a fixed address.

Each choice “applies settings to align with user needs” — tailor them to the system's role (objective 2).

Storage & Partitioning

- A disk is divided into partitions; each partition holds a filesystem.
- Common Linux filesystems: ext4 (general-purpose default) and xfs (Rocky's default, strong with large files).
- Swap is disk space used as overflow when RAM fills up.
- Partitioning can be guided (the installer picks a layout) or manual (you control every partition).
- UEFI systems need an EFI System Partition, and often a separate /boot.

Modify Storage Directories

Linux uses one directory tree (the Filesystem Hierarchy Standard). Key directories can sit on their own partitions to match how the system is used:

- / (root) — the top of the tree; everything hangs off of it.
- /home — user files — often separate so users can't fill the system disk.
- /var — logs, databases, and web content — separated on servers to contain growth.
- /boot — the kernel and bootloader files needed to start the system.
- /etc, /tmp — system configuration and temporary files.

LVM (Logical Volume Manager) lets you resize volumes and add storage later — flexibility that matters as needs change. Choosing this layout is objective 3.

Users & Authentication

- Set a strong root (administrator) password — root can do anything on the system.
- Create a standard user and grant it sudo for everyday administration.
- Avoid logging in as root; many modern distributions disable direct root login by default.
- A strong password plus a separate admin account is your first line of defense.

This is the same su / sudo model from Lesson 2 — set it up correctly at install time.

Software Selection & System Role

- Installers let you pick a role or profile: minimal server, server with GUI, workstation, and more.
- Choose package groups that fit the job — for example a web server or a virtualization host.
- Prefer a minimal install on servers: fewer packages mean fewer running services and a smaller attack surface.
- You can always add software later with the package manager (apt or dnf).

Install Device Drivers

- In Linux, most drivers are kernel modules that load automatically for detected hardware.
- View and manage modules with `lsmod` (list) and `modprobe` (load); firmware lives in `/lib/firmware`.
- Most hardware works out of the box; some devices (certain GPUs, Wi-Fi, printers) need extra or proprietary drivers.
- In virtual machines, install paravirtualized (VirtIO) drivers for fast network, storage, and I/O.

Installing the drivers a host needs to operate is objective 4.

LESSON 5

After the Install



GUI or Headless?

- Servers often run with no GUI to save resources and shrink the attack surface.
- A desktop is convenient on daily-use systems and can be added or removed freely.
- Display server: Wayland (modern, secure) or X11 (older, being phased out).
- Desktop environments: GNOME and KDE Plasma (full-featured); Xfce and LXDE (lightweight).
- You can install through a graphical installer or a text/CLI installer — same result.

Post-Installation Steps

- Apply all available updates immediately (apt update && upgrade, or dnf upgrade).
- Configure the firewall (ufw or firewalld) and confirm SELinux or AppArmor is enabled.
- Create additional users and groups; connect to network storage and services.
- Run any configuration scripts the system's role requires.
- Automate repeatable installs: Kickstart answer files (Red Hat) and cloud-init (cloud VMs).

LIVE DEMO

On Ubuntu: `sudo apt update && sudo apt upgrade`.

Security Focus



Secure It From the First Boot

- Set a strong admin password and use a standard user with sudo instead of root.
- Choose a minimal install — fewer packages, smaller attack surface.
- Apply updates immediately after installation.
- Keep the firewall and SELinux/AppArmor enabled.
- Disable direct root login over SSH.

Why it matters: an exposed, unpatched server can be compromised within minutes. A secure baseline set at install time is far easier — and far safer — than cleaning up after a breach.

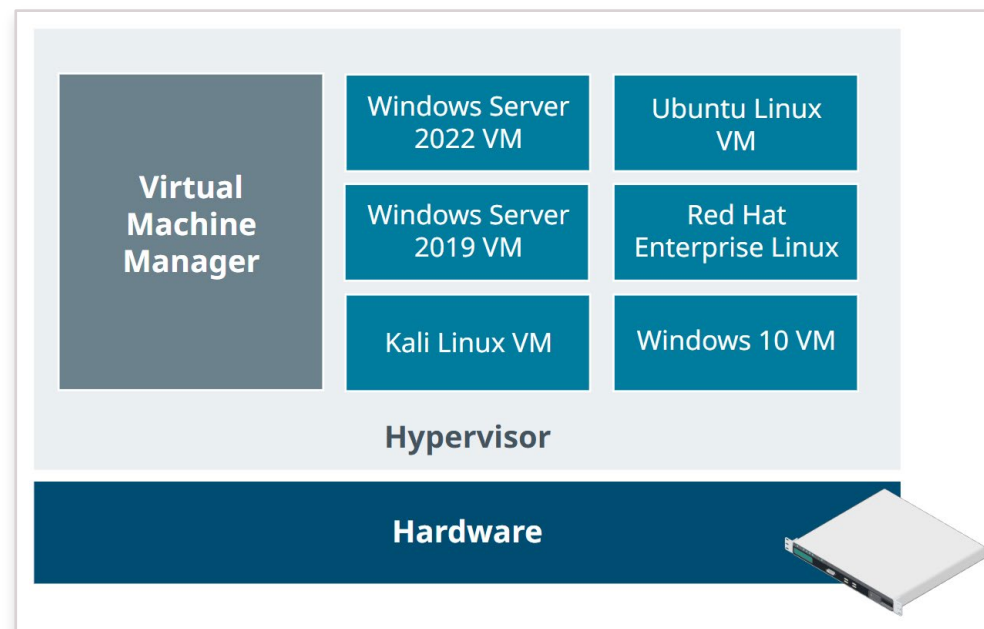
Distro Spotlight: Rocky 10 vs Ubuntu 24.04

	Rocky Linux 10	Ubuntu Server 24.04
Family	Red Hat (RHEL)	Debian
Installer	Anaconda (graphical)	Subiquity (text-based)
Packages	dnf / .rpm	apt / .deb
Default filesystem	xfs	ext4
Firewall	firewalld	ufw
Security module	SELinux	AppArmor

You'll install and work with both this term, so you become fluent across the two families.

Linux in the NETLAB+ Pod

- Your labs run in NETLAB+ on a Proxmox virtualization backend.
- Each student gets a private pod with both distros — Rocky Linux 10 and Ubuntu Server 24.04.
- Pods use snapshots, so you can experiment freely and reset to a clean state.
- You connect through the browser — nothing to install on your own computer.
- *See the Lab 1 — Netlabs Access page for login and scheduling.*



Your Lab: Installing Linux

NETLAB+ Lab — in your pod this week, you will:

- Verify the host's hardware requirements.
- Boot the installer and step through language, time, and network.
- Plan a partition and storage layout and set mount points.
- Create a standard administrative user with sudo.
- Complete the install, apply updates, and confirm a clean boot to login.

Schedule your pod from the Netlabs portal; the full step-by-step is in the lab document.

Key Takeaways

- Linux runs the servers, cloud, and infrastructure you'll support — and it's managed from the command line.
- Bash basics, man pages, editors, and sudo are the everyday tools; get comfortable with them now.
- Nearly every distribution is Debian-based or Red Hat-based; this course uses both.
- A good install starts with checking hardware and choosing current, trustworthy media.
- Storage layout and a non-root admin user are decisions that shape security and maintainability.
- Secure the system from the first boot, then finish with updates and post-install tasks.

Questions?

Next: schedule your NETLAB+ pod and begin the installation lab.

